

C++ Programming

Day 7: 객체 복사와 자원 관리

복사 생성자, 얕은 복사, 깊은 복사, 복사 대입, Rule of Three, RAII

Contents

1	학습 목표	2
2	복사 생성자	2
2.1	왜 참조를 사용하는가	2
2.2	왜 <code>const</code> 를 사용하는가	3
2.3	전체 예제	3
3	얕은 복사	4
3.1	동적 메모리를 가진 클래스	4
3.2	기본 복사 생성자의 동작	4
3.3	값 변경 문제	4
3.4	이중 해제 문제	5
3.5	얕은 복사 전체 예제	5
4	깊은 복사	6
4.1	전체 예제	6
5	복사 대입 연산자	7
5.1	복사 생성과 복사 대입 구분	7
5.2	기본 형태	7
5.3	기존 메모리를 정리해야 하는 이유	7
5.4	자기 대입 문제	8
5.5	전체 예제	8
6	Rule of Three와 RAII	9
6.1	Rule of Three	9
6.2	RAII	9
6.3	왜 멤버를 <code>private</code> 으로 두는가	10
6.4	<code>const</code> 멤버 함수	10
6.5	임시 포인터를 사용하는 이유	10
6.6	전체 예제	11
7	종합 정리	12
8	종합 실습	13
8.1	실습 완성 코드	13
9	실습 파일 구성	14

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 복사 생성자가 호출되는 상황 구분하기
- 컴파일러가 만드는 기본 복사 생성자의 동작 이해하기
- 얕은 복사에서 포인터 주소가 공유되는 이유 설명하기
- 깊은 복사를 사용하여 독립적인 힙 메모리 만들기
- 복사 생성자와 복사 대입 연산자의 차이 구분하기
- 자기 대입 문제를 검사하고 안전하게 처리하기
- Rule of Three의 세 함수를 함께 구현하기
- RAII를 사용하여 객체 수명과 자원 수명 연결하기
- `private` 멤버와 `const` 멤버 함수 사용하기

2. 복사 생성자

복사 생성자(copy constructor)는 이미 존재하는 객체를 이용하여 새로운 객체를 만들 때 호출되는 특별한 생성자이다. 일반적인 형태는 다음과 같다.

```
1 ClassName(const ClassName& other)
2 {
3 }
```

예를 들어 `Book` 객체 `b1`을 이용하여 새로운 객체 `b2`를 만들면 복사 생성자가 호출된다.

```
1 Book b1("C++");
2 Book b2 = b1;
```

다음 표현도 같은 의미이다.

```
1 Book b2(b1);
```

2.1. 왜 참조를 사용하는가

복사 생성자의 매개변수는 일반적으로 다음과 같이 참조로 받는다.

```
1 Book(const Book& other)
```

만약 값으로 받는다면 매개변수 `other`를 만들기 위해 다시 복사 생성자가 필요하다. 그 복사 생성자를 호출하려면 또 다른 복사가 필요하므로 무한 재귀가 발생한다.

```
1 Book(Book other)
2 {
3 }
```

따라서 복사 생성자는 원본 객체를 참조로 받아야 한다.

2.2. 왜 const를 사용하는가

const는 복사 과정에서 원본 객체를 변경하지 않겠다는 뜻이다. 또한 const 객체도 복사할 수 있게 한다.

```
1 const Book b1("C++");
2 Book b2 = b1;
```

핵심 포인트

복사 생성자의 표준 형태는 `ClassName(const ClassName& other)`이다. 참조는 불필요한 복사를 막고, const는 원본 객체를 보호한다.

2.3. 전체 예제

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 class Book
7 {
8 public:
9     string title;
10
11     Book(string title)
12     {
13         this->title = title;
14     }
15
16     Book(const Book& other)
17     {
18         cout << "Book Copied" << endl;
19         this->title = other.title;
20     }
21
22     void showInfo() const
23     {
24         cout << "Title: " << this->title << endl;
25     }
26 };
27
28 int main()
29 {
30     Book b1("C++");
31     Book b2 = b1;
32
33     b1.title = "Python";
34
35     b1.showInfo();
36     b2.showInfo();
37
38     return 0;
39 }
```

`std::string`은 내부적으로 안전한 복사를 제공한다. 따라서 `b1.title`을 변경해도 `b2.title`은 함께 바뀌지 않는다.

3. 얕은 복사

얕은 복사(shallow copy)는 객체가 가진 멤버 값을 그대로 복사한다. 일반적인 정수나 문자열에서는 문제가 드러나지 않을 수 있지만, 포인터 멤버에서는 주소 자체가 복사된다.

3.1. 동적 메모리를 가진 클래스

```

1 class Student
2 {
3 public:
4     int* score;
5
6     Student(int value)
7     {
8         score = new int(value);
9     }
10
11     ~Student()
12     {
13         delete score;
14     }
15 };

```

`score`는 정수 값 자체가 아니라 힙 메모리의 주소를 저장한다.

`score ----> [100]`

3.2. 기본 복사 생성자의 동작

사용자가 복사 생성자를 작성하지 않으면 컴파일러가 기본 복사 생성자를 만든다. 개념적으로 다음과 비슷하게 동작한다.

```

1 Student(const Student& other)
2 {
3     score = other.score;
4 }

```

이 코드는 새로운 정수 메모리를 만들지 않는다. 단지 원본 포인터에 저장된 주소를 그대로 복사한다.

`s1.score ----\
 ---> [100]
s2.score ----/`

두 객체는 하나의 힙 메모리를 공유한다.

3.3. 값 변경 문제

```

1 Student s1(100);
2 Student s2 = s1;
3
4 *s2.score = 999;

```

`s1.score`와 `s2.score`가 같은 메모리를 가리키므로 두 객체에서 모두 999가 출력된다.

3.4. 이중 해제 문제

프로그램이 끝나면 두 객체의 소멸자가 각각 호출된다.

```
1 delete s2.score;
2 delete s1.score;
```

하지만 두 포인터가 같은 메모리를 가리키므로 같은 메모리를 두 번 해제하게 된다. 이것을 이중 해제(double delete)라고 하며, 프로그램의 동작은 정의되지 않는다.

핵심 포인트

포인터 멤버를 기본 복사하면 포인터가 가리키는 값이 아니라 주소가 복사된다. 따라서 값 공유, dangling 포인터, 이중 해제 문제가 발생할 수 있다.

3.5. 얽은 복사 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Student
6 {
7 public:
8     int* score;
9
10    Student(int value)
11    {
12        score = new int(value);
13    }
14
15    ~Student()
16    {
17        delete score;
18    }
19
20    void showInfo() const
21    {
22        cout << *score << endl;
23    }
24 };
25
26 int main()
27 {
28     Student s1(100);
29     Student s2 = s1;
30
31     *s2.score = 999;
32
33     s1.showInfo();
34     s2.showInfo();
35
36     return 0;
37 }
```

이 예제는 얽은 복사의 문제를 보여 주기 위한 코드이다. 실제 프로그램에서는 그대로 사용하면 안 된다.

4. 깊은 복사

깊은 복사(deep copy)는 포인터의 주소만 복사하지 않고 새로운 힙 메모리를 만든 뒤 원본 값을 복사한다.

```
1 Book(const Book& other)
2 {
3     price = new int(*other.price);
4 }
```

`other.price`는 원본 객체의 포인터이고, `*other.price`는 그 포인터가 가리키는 실제 정수 값이다.

```
1 new int(*other.price)
```

위 코드는 원본 값으로 초기화된 새로운 정수 공간을 힙에 만든다.

b1.price ----> [30000]

b2.price ----> [30000]

두 값은 같지만 메모리 주소는 서로 다르다. 따라서 한 객체의 값을 변경해도 다른 객체에는 영향을 주지 않는다.

4.1. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Book
6 {
7 public:
8     int* price;
9
10    Book(int price)
11    {
12        this->price = new int(price);
13    }
14
15    Book(const Book& other)
16    {
17        cout << "Book Deep Copied" << endl;
18        this->price = new int(*other.price);
19    }
20
21    ~Book()
22    {
23        delete this->price;
24    }
25
26    void showInfo() const
27    {
28        cout << *this->price << endl;
29    }
30 };
31
32 int main()
33 {
34     Book b1(30000);
35     Book b2 = b1;
36 }
```

```

37     *b2.price = 50000;
38
39     b1.showInfo();
40     b2.showInfo();
41
42     return 0;
43 }

```

출력 결과는 다음과 같다.

```

Book Deep Copied
30000
50000

```

5. 복사 대입 연산자

복사 생성자는 새 객체를 만들 때 사용된다. 이미 존재하는 객체에 다른 객체를 대입할 때는 복사 대입 연산자(copy assignment operator)가 호출된다.

5.1. 복사 생성과 복사 대입 구분

```

1 Book b1(30000);
2 Book b2 = b1; // Copy constructor
3
4 Book b3(50000);
5 b3 = b1; // Copy assignment operator

```

첫 번째 경우에는 b2가 새로 생성된다. 두 번째 경우에는 b3가 이미 존재하므로 기존 자원을 정리하고 새로운 값을 복사해야 한다.

5.2. 기본 형태

```

1 Book& operator=(const Book& other)
2 {
3     return *this;
4 }

```

반환형이 Book&인 이유는 현재 객체 자신을 참조로 반환하여 연속 대입을 가능하게 하기 위해서이다.

```

1 b3 = b2 = b1;

```

5.3. 기존 메모리를 정리해야 하는 이유

이미 존재하는 객체는 자신이 소유한 힙 메모리를 가지고 있다. 새 주소를 바로 덮어쓰면 기존 주소를 잃어버려 메모리 누수가 발생한다.

잘못된 형태는 다음과 같다.

```

1 this->price = new int(*other.price);

```

기존 메모리를 정리한 뒤 새 메모리를 연결해야 한다. 더 안전한 순서는 먼저 새 메모리를 확보하고 그 다음 기존 메모리를 해제하는 것이다.

```

1 int* newPrice = new int(*other.price);
2
3 delete this->price;
4 this->price = newPrice;

```

새 메모리 할당에 실패하더라도 기존 데이터가 먼저 사라지지 않는다는 장점이 있다.

5.4. 자기 대입 문제

다음 코드에서는 원본 객체와 대입받는 객체가 같다.

```
1 b1 = b1;
```

이때 `this`와 `&other`는 같은 주소를 가진다. 기존 메모리를 먼저 삭제한 뒤 `other.price`를 읽으면 이미 삭제된 메모리를 역참조하게 된다. 따라서 자기 대입 여부를 먼저 검사한다.

```
1 if (this == &other)
2 {
3     return *this;
4 }
```

5.5. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Book
6 {
7 public:
8     int* price;
9
10    Book(int price)
11    {
12        this->price = new int(price);
13    }
14
15    Book(const Book& other)
16    {
17        this->price = new int(*other.price);
18    }
19
20    Book& operator=(const Book& other)
21    {
22        if (this == &other)
23        {
24            return *this;
25        }
26
27        int* newPrice = new int(*other.price);
28
29        delete this->price;
30        this->price = newPrice;
31
32        cout << "Copy Assignment" << endl;
33
34        return *this;
35    }
36
37    ~Book()
38    {
39        delete this->price;
40    }
41
42    void showInfo() const
43    {
44        cout << *this->price << endl;
```

```

45     }
46 };
47
48 int main()
49 {
50     Book b1(30000);
51     Book b2(50000);
52
53     b2 = b1;
54
55     *b2.price = 100000;
56
57     b1.showInfo();
58     b2.showInfo();
59
60     return 0;
61 }

```

출력 결과는 다음과 같다.

```

Copy Assignment
30000
100000

```

6. Rule of Three와 RAII

동적 메모리처럼 직접 관리해야 하는 자원을 가진 클래스는 복사와 소멸 동작을 함께 설계해야 한다.

6.1. Rule of Three

다음 세 함수 중 하나를 직접 구현해야 한다면 나머지 둘도 필요할 가능성이 높다.

1. 소멸자
2. 복사 생성자
3. 복사 대입 연산자

이를 Rule of Three라고 한다.

```

1 ~ClassName();
2 ClassName(const ClassName& other);
3 ClassName& operator=(const ClassName& other);

```

소멸자만 구현하면 복사 과정에서 주소가 공유될 수 있다. 복사 생성자만 구현하면 이미 존재하는 객체끼리의 대입에서 얇은 복사가 발생할 수 있다. 복사 대입 연산자만 구현하면 새 객체를 복사 생성할 때 기본 복사가 사용될 수 있다. 따라서 세 함수를 하나의 세트로 생각해야 한다.

6.2. RAII

RAII는 Resource Acquisition Is Initialization의 약자이다. 핵심은 객체가 생성될 때 자원을 획득하고 객체가 소멸될 때 자원을 자동으로 해제하도록 만드는 것이다.

```

1 NumberBox(int value)
2 {
3     number = new int(value);
4 }
5
6 ~NumberBox()

```

```

7 {
8     delete number;
9 }

```

객체의 수명과 힙 메모리의 수명이 연결된다.

객체 생성 -> 자원 획득

객체 사용 -> 자원 유지

객체 소멸 -> 자원 해제

RAII는 동적 메모리뿐 아니라 파일, 네트워크 연결, 락과 같은 자원 관리에도 사용된다.

6.3. 왜 멤버를 private으로 두는가

포인터 멤버가 public이면 외부 코드가 객체의 자원 관리 규칙을 깨뜨릴 수 있다.

```

1 delete box.number;
2 box.number = nullptr;

```

이런 조작은 이중 해제나 메모리 누수를 만들 수 있다. 따라서 자원 포인터는 private으로 숨기고, 필요한 기능만 public 멤버 함수로 제공한다.

```

1 private:
2     int* number;
3
4 public:
5     void setNumber(int value);
6     void showInfo() const;

```

이것은 데이터와 구현 세부 사항을 숨기는 캡슐화의 한 예이다.

6.4. const 멤버 함수

멤버 함수 뒤의 const는 이 함수가 객체의 상태를 변경하지 않겠다는 약속이다.

```

1 void showInfo() const
2 {
3     cout << *number << endl;
4 }

```

읽기 전용 함수에 const를 붙이면 const 객체에서도 호출할 수 있다.

```

1 const NumberBox box(10);
2 box.showInfo();

```

반면 값을 변경하는 함수에는 const를 붙이지 않는다.

```

1 void setNumber(int value)
2 {
3     *number = value;
4 }

```

6.5. 임시 포인터를 사용하는 이유

복사 대입에서 다음 순서를 사용하면 새 메모리 할당이 성공한 뒤에만 기존 자원을 버리게 된다.

```

1 int* newNumber = new int(*other.number);
2
3 delete number;
4 number = newNumber;

```

newNumber는 새 자원을 임시로 보관한다. 모든 준비가 끝난 뒤 number가 그 주소를 넘겨받는다. 이때 포인터 변수만 사라지고 실제 힙 메모리는 number가 계속 소유한다.

6.6. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 class NumberBox
6 {
7 private:
8     int* number;
9
10 public:
11     NumberBox(int value)
12     {
13         number = new int(value);
14     }
15
16     NumberBox(const NumberBox& other)
17     {
18         number = new int(*other.number);
19     }
20
21     NumberBox& operator=(const NumberBox& other)
22     {
23         if (this == &other)
24         {
25             return *this;
26         }
27
28         int* newNumber = new int(*other.number);
29
30         delete number;
31         number = newNumber;
32
33         return *this;
34     }
35
36     ~NumberBox()
37     {
38         delete number;
39     }
40
41     void setNumber(int value)
42     {
43         *number = value;
44     }
45
46     int getNumber() const
47     {
48         return *number;
49     }
50
51     void showInfo() const
52     {
53         cout << "Number: " << *number << endl;
54     }
55 };
56
57 int main()
58 {
```

```

59     NumberBox box1(10);
60     NumberBox box2 = box1;
61     NumberBox box3(30);
62
63     box2.setNumber(20);
64     box3 = box1;
65     box3.setNumber(40);
66
67     box1.showInfo();
68     box2.showInfo();
69     box3.showInfo();
70
71     return 0;
72 }

```

출력 결과는 다음과 같다.

```

Number: 10
Number: 20
Number: 40

```

세 객체는 서로 다른 힙 메모리를 소유한다. 따라서 한 객체의 값을 바꾸어도 다른 객체의 값은 변하지 않는다.

7. 종합 정리

핵심 포인트

- 복사 생성자는 기존 객체로 새 객체를 만들 때 호출된다.
- 기본 복사는 멤버별 복사를 수행하며 포인터 멤버에서는 주소가 복사된다.
- 얇은 복사는 값 공유와 이중 해제 문제를 만들 수 있다.
- 깊은 복사는 새 힙 메모리를 할당하고 원본 값을 복사한다.
- 복사 대입 연산자는 이미 존재하는 객체 사이의 대입을 처리한다.
- 복사 대입에서는 자기 대입 검사와 기존 자원 정리가 필요하다.
- 소멸자, 복사 생성자, 복사 대입 연산자는 Rule of Three로 함께 설계한다.
- RAII는 객체의 수명과 자원의 수명을 연결한다.
- 자원 포인터는 `private`으로 숨기고 읽기 함수에는 `const`를 붙인다.

8. 종합 실습

실습

Book 클래스를 작성한다.

- private 멤버로 `int* price`를 가진다.
- 생성자에서 가격을 저장할 힙 메모리를 만든다.
- 깊은 복사를 수행하는 복사 생성자를 작성한다.
- 자기 대입을 처리하는 복사 대입 연산자를 작성한다.
- 소멸자에서 힙 메모리를 해제한다.
- `setPrice()`와 `showInfo()` `const`를 작성한다.
- 복사 생성과 복사 대입 후 객체들이 독립적으로 동작하는지 확인한다.

8.1. 실습 완성 코드

```

1 #include <iostream>
2
3 using namespace std;
4
5 class Book
6 {
7 private:
8     int* price;
9
10 public:
11     Book(int price)
12     {
13         this->price = new int(price);
14     }
15
16     Book(const Book& other)
17     {
18         this->price = new int(*other.price);
19     }
20
21     Book& operator=(const Book& other)
22     {
23         if (this == &other)
24         {
25             return *this;
26         }
27
28         int* newPrice = new int(*other.price);
29
30         delete this->price;
31         this->price = newPrice;
32
33         return *this;
34     }
35
36     ~Book()
37     {

```

```
38     delete this->price;
39 }
40
41 void setPrice(int price)
42 {
43     *this->price = price;
44 }
45
46 void showInfo() const
47 {
48     cout << "Price: " << *this->price << endl;
49 }
50 };
51
52 int main()
53 {
54     Book b1(30000);
55     Book b2 = b1;
56     Book b3(50000);
57
58     b2.setPrice(40000);
59     b3 = b1;
60     b3.setPrice(100000);
61
62     b1.showInfo();
63     b2.showInfo();
64     b3.showInfo();
65
66     return 0;
67 }
```

9. 실습 파일 구성

실습

Day 7 실습 파일은 다음과 같이 관리한다.

- practice/p01_copy_constructor.cpp
- practice/p02_shallow_copy.cpp
- practice/p03_deep_copy.cpp
- practice/p04_copy_assignment.cpp
- practice/p05_rule_of_three.cpp